

HIERARCHICAL SKILL EVOLUTION FOR SOFTWARE-ENGINEERING AGENTS

Anonymous authors

Paper under double-blind review

ABSTRACT

Software-engineering agents generate rich traces—issue interpretations, file searches, edits, test runs, failures, and recoveries—that are discarded after each benchmark trial. The obvious move is to recycle these traces into reusable text skills, but the obvious move is also where most systems quietly cheat: raw trace retrieval smuggles task-specific solutions into future prompts, and one-shot “summarize the trajectory” pipelines emit over-specific or actively harmful instructions that no one audits. Either way, apparent gains come from memorization or noise rather than transfer. We present HSE, a verifier-grounded framework that evolves external skills for *frozen* coding agents while making leakage structurally hard. HSE refuses to call every trace artifact a skill: it separates single-task evidence from reusable skills, promotes evidence through a hierarchy of repo-level and cross-repo failure-mode skills, and trains separate Skill Writer and Skill Evaluator policies behind a fixed harness policy. The design is built around one hard information boundary. During SWEGym training the official verifier supplies outcome labels, evaluator calibration, and validation gates; at downstream evaluation the verifier never touches the update path—no hidden test, gold patch, or future outcome may alter a skill. We instantiate the framework in a runnable system (Pi harness, OpenAI-compatible models, Harbor/E2B execution, versioned skill archives with promotion logs and validation gates) and specify an evaluation protocol over a 2×2 executor matrix—GLM-5.2 and GPT-5.5 $\times$ Pi and Claude-style harnesses—that isolates model transfer, harness transfer, and joint transfer, with SWE-bench Verified and DeepSWE as in-domain targets and Terminal-Bench 2.0/2.1 as an out-of-domain extrapolation test, all under a shared sandbox and acceptance rule.

1 INTRODUCTION

Large language model agents can repair real software bugs: they read a repository, edit code, run tests, and recover from failures. Every one of those attempts—successful or not—produces information about how to localize, edit, validate, and recover on the next task. Almost all of it is thrown away the moment the benchmark scores the trial. Recycling it into external *skills*—compact textual procedures injected into future prompts—is an attractive way to make a frozen agent better without retraining its weights.

The attraction hides a trap. A raw trajectory is full of things that must not leak forward: concrete file paths, the exact shape of a gold patch, transient environment errors, and incidental identifiers of the benchmark instance. Inject those into a future prompt and the system improves by *memorization*, not transfer—and on a held-out benchmark this is indistinguishable from progress until someone audits the prompt. The opposite failure is just as common: a one-shot “summarize the trajectory into advice” step yields instructions like “run focused tests” that are too generic to change behavior, or over-specific rules that fire on the wrong task and cause regressions. We argue that most trace-to-skill pipelines sit on one of these two failure modes and report the resulting numbers as if they measured transfer.

The question we take seriously is therefore: *can a coding-agent system learn reusable software-engineering skills from training-time verifier feedback without using test-time verifier feedback and without leaking task-specific solutions?* Answering it requires more than a better summarizer. It

requires (i) a disciplined notion of what is allowed to become a transferable skill, and (ii) a hard, auditable boundary on when the verifier may influence memory.

We propose HSE, **hierarchical skill evolution**. The central design commitment is restraint: *not every trace artifact is a skill*. A task attempt first becomes *task evidence*—a structured record of public signals (touched and edited paths, test commands, diagnostics, selected skills, failure signatures, verifier outcome). Task evidence is never rendered as a deployed skill. A Skill Writer drafts a repo-level skill only from *multiple* evidence records in the *same* repository, and a failure-mode skill only from patterns that *recur across* repositories—localization drift, weak validation, no-diff exits, timeout recovery. A Skill Evaluator then scores each candidate for utility, support, and risk and decides whether to accept, reject, revise, or retain it as memory-only evidence. Every upward step is gated; a single lucky trace cannot mint a global skill.

The boundary is the second commitment, and it is hard rather than aspirational. SWEGym verifiers are available at training time, so we use them to label outcomes, calibrate the evaluator, and gate candidate states on a repo-isolated validation split. On SWE-bench Verified and other downstream benchmarks, no official verifier label, hidden test result, gold patch, or future task outcome may influence a skill update—full stop. This is what makes a transfer claim on those benchmarks meaningful rather than a restatement of leaked supervision.

Contributions.

- We recast software-agent skill evolution as a hierarchical, evidence-gated process that cleanly separates task evidence, repo-level skills, cross-repo failure-mode skills, and the writer/evaluator policies that produce and police them.
- We define a verifier-boundary protocol that licenses training-time use of SWEGym verifier outcomes while forbidding any verifier signal on the test-time update path, turning leakage from a reviewer’s worry into a structural property of the system.
- We instantiate the framework end to end for the Pi coding-agent harness with an OpenAI-compatible model interface, Harbor/E2B execution, versioned skill archives, promotion logs, validation gates, and a deterministic evaluator shell that can be swapped for a learned policy without changing the loop contract.
- We specify an evaluation protocol over a GLM-5.2/GPT-5.5 $\times$ Pi/Claude-style matrix that separates within-cell evolution, model transfer, and harness transfer across SWE-bench Verified, DeepSWE, and an out-of-domain Terminal-Bench 2.0/2.1 extrapolation, treating exact-task memory only as an explicitly labeled adaptation upper bound.

2 PROBLEM SETTING

Tasks and agents. A software-engineering task x consists of a natural-language issue, a repository snapshot, and a public execution environment. A frozen coding agent M interacts with a harness H by issuing file reads, searches, edits, shell commands, and a final answer. A trajectory τ records the observable history: prompts, tool calls, outputs, diffs, public tests, and runtime diagnostics. M and H are fixed throughout; the only thing that evolves is the external skill state.

Training-time verifier. During SWEGym training we hold a verifier V mapping a completed trial to a reward $r \in \{0, 1\}$ or a diagnostic outcome. V supervises training-time labels and validation gates. It is deliberately *not* a writer of skills and *not* a source of hidden solutions. At downstream test time V is unavailable for any memory update; it is invoked only after the protocol completes, to report final benchmark scores.

Skill state. The external state S holds structured evidence and accepted skills in four deployed categories:

- **Task evidence** records what happened on one task. It is not a reusable skill and is never deployed as one.
- **Repo skills** summarize repeated, evidence-gated procedures within a single repository.

Table 1: Skill-evolution objects and update clocks. The main experiments freeze all training-derived transferable state before any downstream evaluation.

Level	Object	Update clock	Evidence requirement
L0	Rollout trace	Every task attempt	Public trajectory and runtime outputs
L1	Task evidence	Every task attempt	One task trace; verifier label (train only)
L2	Repo skill candidate	Same-repo batch	Multiple evidence records, one repo
L3	Failure-mode candidate	Cross-repo trigger	Repeated repo summaries across repos
L4	Skill Writer policy	Slow policy window	Aggregated accept/reject histories
L5	Skill Evaluator policy	Slow policy window	Verifier-calibrated false accepts/rejects

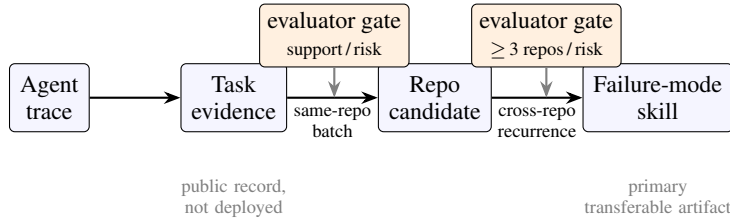


Figure 1: The promotion pipeline. A trace becomes task evidence, then a repo-level candidate, then a cross-repo failure-mode skill. Task evidence is never deployed directly; each upward step passes an evaluator gate that checks support, risk, and validation. Repo promotion needs repeated same-repo evidence; failure-mode promotion needs at least three distinct repositories.

- **Failure-mode skills** summarize cross-repo recovery strategies for recurring agent failures. These are the primary transferable artifact.
- **General seed skills** provide weak process priors for common stages (reproduce, localize, edit, validate, recover).

The writer policy π_w drafts evidence summaries and candidate skills; the evaluator policy π_e assigns decisions, proxy rewards, confidence, and risk flags.

3 HIERARCHICAL SKILL EVOLUTION

3.1 A HIERARCHY WITH DIFFERENT UPDATE CLOCKS

HSE updates different abstraction levels at different rates. Low levels change quickly because a mistake there touches only local evidence; high levels change slowly because a mistake there propagates into every future task. Table 1 lists the objects, their clocks, and the evidence each promotion requires.

Task evidence stores public fields only—task id, repo, problem summary, reward, previous reward, case label, exception type, selected skills, touched and edited paths, public test commands, failure signature, and diagnostic signals. It feeds higher-level aggregation but is never exported as a reusable `SKILL.md`.

3.2 WRITER AND EVALUATOR: GENERATION VERSUS GATEKEEPING

The writer turns clusters of evidence into structured candidates. From a cluster of same-repo evidence it emits a repo candidate with fields name, trigger, evidence gate, actions, validation hint,

abort condition, and support summary. From a cross-repo cluster it emits a process-level failure-mode candidate with a failure signature, trigger, actions, stop condition, and support summary:

$$c_{\text{repo}} = f_w(E_{\text{repo}}), \quad c_{\text{fail}} = f_w(E_{\text{cross-repo}}).$$

The structure is not decoration: it forces every candidate to expose its trigger, support, actions, and risks so the evaluator can inspect them before anything is deployed.

The evaluator is the gatekeeper that separates generation from deployment:

$$\pi_e(c, E) \rightarrow (d, \hat{r}, \rho, z),$$

with decision $d \in \{\text{ACCEPT}, \text{REJECT}, \text{REVISE}, \text{MEMORY-ONLY}\}$, proxy reward $\hat{r}$, confidence ρ , and risk flags z . Hard safety risks—hidden verifier details, oracle or exact patches, task-id memorization, private sandbox paths—force rejection regardless of apparent utility. Repo candidates require repeated same-repo support; failure-mode candidates require support from multiple distinct repositories. A single repo can never originate a failure-mode skill.

We deliberately ship a *deterministic* evaluator shell with explicit thresholds and risk checks. This is an interface choice, not a limitation: it makes the accept/reject contract auditable today, while exposing exactly the same call signature that a learned evaluator—trained on SWEgym calibration events—can later occupy without disturbing the rest of the loop.

3.3 A FIXED HARNESS POLICY THE LEARNED POLICIES CANNOT REWRITE

The learned writer and evaluator are explicitly forbidden from inventing their own reward labels, update clocks, or promotion budgets. A fixed harness policy maps verifier transitions and runtime diagnostics to case labels:

- **Strong positive:** previous attempt failed, current attempt passed.
- **Weak positive:** current attempt passed without a paired improvement, or a previously passing attempt stays passing.
- **Strong negative:** previous attempt passed, current attempt failed.
- **Weak negative:** current attempt failed without a paired regression.
- **Diagnostic:** missing reward, timeout, setup error, verifier error, sandbox failure, or provider/runtime failure.

Cold-start solved traces are weak positives; unsolved traces are weak negatives unless a diagnostic is present. Strong positives may add support to a candidate but cannot by themselves create a global skill. Strong negatives prioritize rollback, trigger narrowing, and evaluator false-accept penalties. Diagnostics are kept out of semantic edit skills entirely and used only to shape runtime or recover/validate safeguards.

3.4 TRAINING LOOP AND THE VERIFIER BOUNDARY

Training alternates rollout, evidence extraction, candidate generation, evaluator gating, and validation (Figure 2, top lane). The verifier supervises case labels, calibrates the evaluator, and gates candidate states—and nothing more. At downstream evaluation (bottom lane) the writer, evaluator, and transferable skills are frozen and only public trace signals enter the update path.

Default update hyperparameters. Unless stated otherwise, repo updates use same-repo batches of 5 evidence events, failure-mode updates trigger every 4 repo updates, a failure-mode promotion requires at least 3 supporting repositories, and policy updates are capped at 3 bullet edits per policy window. Each repo batch emits at most one repo candidate; each failure-mode trigger promotes at most two accepted failure-mode skills. These values are the harness defaults and are treated as fixed protocol hyperparameters, not tuned per benchmark.

3.5 VALIDATION GATE

Candidate skill states are scored on a repo-isolated SWEgym validation set via

$$\text{effective_resolved_rate} = \frac{\# \text{ resolved valid trials}}{\# \text{ effective valid trials}}, \quad \text{diagnostic_rate} = \frac{\# \text{ diagnostic trials}}{\# \text{ total trials}}.$$

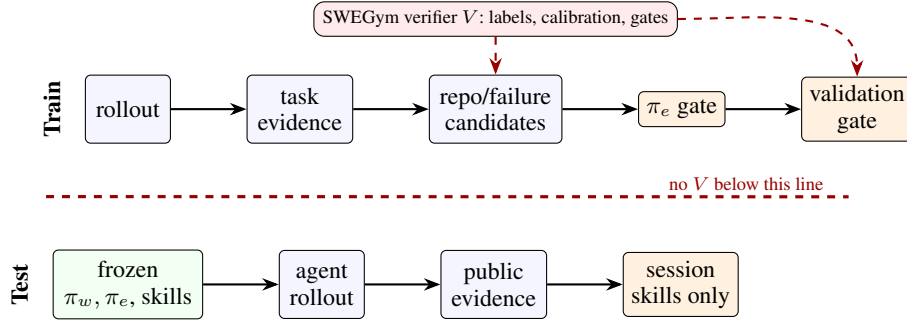


Figure 2: The verifier boundary. **Top (train):** verifier V supplies outcome labels, evaluator calibration, and validation gates. **Bottom (test):** the writer, evaluator, and transferable skills are frozen and only public trace signals update memory. V appears in the training lane and in final reporting only—never on the downstream update path.

A candidate state is accepted only if it has enough effective validation trials, stays under the diagnostic-rate threshold, and does not regress beyond a fixed tolerance relative to the current accepted state. Otherwise the candidate version is rejected or rolled back and the decision is logged. The gate exists precisely to stop a plausible-looking candidate from silently trading pass rate for regressions.

4 EXPERIMENTS

We report the experimental *settings* in full and leave numeric cells as – until the corresponding runs complete; every table is structured so a reader sees the baseline, the HSE result, and their difference side by side rather than as disconnected rows.

4.1 BENCHMARKS

In-domain targets. The primary downstream benchmark is **SWE-bench Verified**, the human-verified subset of SWE-bench. We additionally evaluate on **DeepSWE**, a harder issue-resolution suite, to test that gains are not an artifact of a single benchmark. On both, no official verifier label, hidden test, or gold patch enters the skill-update path during the evaluated run; the official verifier is invoked only afterward to report resolved count and rate.

Out-of-domain extrapolation. We treat **Terminal-Bench 2.0 and 2.1** as an extrapolation test: the agent faces general command-line tasks rather than repository patching, but runs through the same Harbor/E2B wrapper. This isolates whether the cross-repo failure-mode skills—reproduce, localize, validate, recover—survive a shift away from the issue-resolution distribution they were evolved on, or whether they silently degrade.

Evolution source. All transferable state is evolved on a 500-task SWEgym subset under a repo-isolated split: validation repositories are held out from training, with the validation size chosen closest to 5% of the subset, yielding 474 training and 26 validation tasks. The SWEgym verifier is available only in this phase.

4.2 MODELS AND HARNESSES

Executor matrix. We evaluate two executor models, GLM-5.2 (glm52) and GPT-5.5, each under two agent harnesses, Pi and a Claude-style coding-agent harness. All four cells share the same Harbor/E2B sandbox interface, task selection, task ordering, timeout budget, validation gate, and no-test-verifier update rule. Where model APIs differ we route through one OpenAI-compatible provider abstraction and record the endpoint type in the run config.

Table 2: Evaluation protocols. Exact-task skills are reported only as an adaptation upper bound and never used to support a transfer claim.

Setting	Exact task memory	Test-time update	Purpose
No Skills	✗	✗	Frozen-agent baseline
Public SWE Skills	✗	✗	Static public-skill baseline
Raw Trace Retrieval	✗	✗	Does raw trajectory memory suffice?
One-shot StageSkill	✗	✗	Ungated trace-to-skill summarization
Frozen Failure Skills	✗	✗	SWEGym-trained transferable skills
Frozen π_w/π_e + online repo	✗	public only	Verifier-free online evolution
Same-budget Retry	✗	✗	Controls for extra attempts/compute
Exact Task Skills	✓	optional	Adaptation upper bound only

Three transfer questions. The 2×2 matrix isolates three questions that prior work routinely conflates. **Model transfer:** skills and policies evolved with one model are evaluated with the other under a fixed harness. **Harness transfer:** skills evolved under one harness are evaluated under the other with a fixed model. **Joint transfer:** both model and harness change between evolution and evaluation. We report each separately from within-cell evolution, where evolution and evaluation share the same cell.

Controls. The tool surface is fixed within each harness family (read, write, edit, bash, grep, find, ls). Prompts may differ between Pi and Claude-style harnesses, but task order, sandbox resources, retry policy, and acceptance rules are held fixed. Every run logs the active skill version, the source and target cell, selected skills, runtime diagnostics, verifier reports, and score reports, so any reported delta is attributable to the skill state rather than to a moving control.

4.3 PROTOCOLS

4.4 METRICS

The primary metric is resolved rate. Because a skill system can raise pass rate while quietly inflating regressions, latency, or prompt cost, we also report resolved-count deltas; paired transition counts ($0 \rightarrow 1$, $1 \rightarrow 0$, $1 \rightarrow 1$, $0 \rightarrow 0$); diagnostic, no-diff, timeout, and weak-validation rates; localization-drift rate; selected-skill read and trigger-match rates; retrieval false-positive rate; and tool-call count, token count, wall-clock time, and validation-command count. A method that wins on resolved rate while losing on the second group has not actually won.

4.5 MAIN RESULTS

Table 3 reports within-cell resolved rate on the two in-domain benchmarks for every model/harness cell, baseline against HSE. Table 4 reports the out-of-domain Terminal-Bench extrapolation. Table 5 reports cross-cell transfer, and Table 6 ablates each mechanism.

5 RELATED WORK

Software-engineering benchmarks. SWE-bench introduced real GitHub issue resolution as a benchmark for language models and coding agents (Jimenez et al., 2024); SWE-bench Verified refines it with human-verified tasks and criteria (OpenAI, 2024). SWEGym provides executable training environments and verifiers for software-engineering agents and is the source of verifier-supervised evolution in our work (Pan et al., 2024). Terminal-Bench extends evaluation to broader terminal workflows beyond repository patching (Merrill et al., 2026), which we use as our out-of-domain extrapolation target.

Table 3: Within-cell resolved rate (%) on the in-domain benchmarks. Each cell evolves and evaluates with the same model/harness. Δ is HSE minus No Skills; positive is better.

Model	Harness	SWE-bench Verified			DeepSWE		
		No Skills	HSE	Δ	No Skills	HSE	Δ
GLM-5.2	Pi	—	—	—	—	—	—
	Claude-style	—	—	—	—	—	—
GPT-5.5	Pi	—	—	—	—	—	—
	Claude-style	—	—	—	—	—	—

Table 4: Out-of-domain extrapolation: resolved rate (%) on Terminal-Bench 2.0/2.1 using failure-mode skills evolved on SWEgym issue resolution. A non-negative Δ indicates the recovery skills transfer past the issue-resolution distribution.

Model	Harness	Terminal-Bench 2.0			Terminal-Bench 2.1		
		No Skills	HSE	Δ	No Skills	HSE	Δ
GLM-5.2	Pi	—	—	—	—	—	—
	Claude-style	—	—	—	—	—	—
GPT-5.5	Pi	—	—	—	—	—	—
	Claude-style	—	—	—	—	—	—

Language-agent memory and reflection. Prior agents reuse verbal feedback, self-reflection, or episodic memory to improve future behavior (Yao et al., 2023; Shinn et al., 2023; Wang et al., 2023). HSE departs from this line in two ways that matter for evaluation integrity: it enforces a benchmark-verifier boundary, and it refuses to treat task-local evidence as a transferable skill.

Skill extraction and text-space optimization. A recent wave of systems distills trajectories into textual skills, optimizes external instructions, or curates skill repositories (Ni et al., 2026; Yang et al., 2026a; Ouyang et al., 2026; Yang et al., 2026b). These methods largely assume the harvested skill is safe to deploy. Our setting is harder and more honest: executable verifiers exist at training time but are forbidden at test-time memory update, and exact-task leakage is treated as the central evaluation risk rather than an afterthought.

6 LIMITATIONS

HSE has real limits. First, verifier-grounded training is only as trustworthy as the verifier and validation split; incomplete public tests can still miscalibrate the evaluator. Second, failure-mode skills can be over-broad when failure signatures are noisy, so we report trigger-match and retrieval-false-positive rates rather than pass rate alone. Third, online test-time evolution is order-sensitive; we report both original-order and random-order robustness. Fourth, the shipped evaluator is deterministic and rule-based—a learned evaluator must be stress-tested against false accepts, false rejects, and prompt bloat before it replaces the shell. Finally, within-cell, model-transfer, harness-transfer, and joint-transfer results answer different questions and must be read separately; collapsing them is exactly the conflation this protocol is designed to prevent.

7 CONCLUSION

We presented HSE, a verifier-grounded approach for evolving external software-engineering skills for frozen coding agents. By promoting public task evidence through repo-level and cross-repo failure-mode gates, separating writer and evaluator policies, and freezing all transferable state behind a hard verifier boundary before downstream evaluation, HSE makes leakage structurally hard instead of merely discouraged. The protocol is built to answer one question cleanly: whether SWEgym

Table 5: Cross-cell transfer on SWE-bench Verified. Source is the cell used for skill evolution; target is the cell used for evaluation. Δ is HSE minus the target cell’s No-Skills baseline.

Source cell	Target cell	Type	Resolved Δ
GLM-5.2 / Pi	GPT-5.5 / Pi	Model	–
GPT-5.5 / Pi	GLM-5.2 / Pi	Model	–
GLM-5.2 / Claude-style	GPT-5.5 / Claude-style	Model	–
GPT-5.5 / Claude-style	GLM-5.2 / Claude-style	Model	–
GLM-5.2 / Pi	GLM-5.2 / Claude-style	Harness	–
GLM-5.2 / Claude-style	GLM-5.2 / Pi	Harness	–
GPT-5.5 / Pi	GPT-5.5 / Claude-style	Harness	–
GPT-5.5 / Claude-style	GPT-5.5 / Pi	Harness	–
GLM-5.2 / Pi	GPT-5.5 / Claude-style	Joint	–
GPT-5.5 / Claude-style	GLM-5.2 / Pi	Joint	–

Table 6: Ablations on SWE-bench Verified. Each row removes one mechanism while holding model, harness, task order, and budget fixed. A negative Δ means the removed mechanism was load-bearing.

Ablation	Question	Resolved Δ
– evaluator gate	Do ungated candidates introduce harmful skills?	–
– validation gate	Does repo-isolated validation prevent regressions?	–
– failure-mode skills	Are cross-repo recovery rules useful?	–
– repo skills	Are same-repo procedures useful online?	–
– policy update windows	Do slow clocks reduce drift and prompt bloat?	–
Raw trace retrieval only	Is structured abstraction necessary?	–

can train reusable skill-writing and skill-selection behavior that transfers in-domain to SWE-bench Verified and DeepSWE and out-of-domain to Terminal-Bench 2.0/2.1, without test-time verifier feedback.

REFERENCES

- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024.
- Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, et al. Terminal-Bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026.
- Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, Dexin Wang, Xiaoxi Jiang, and Guanjun Jiang. Trace2Skill: Distill trajectory-local lessons into transferable agent skills, 2026.
- OpenAI. SWE-bench Verified. <https://openai.com/index/introducing-swe-bench-verified/>, 2024. Human-validated subset of SWE-bench.
- Siru Ouyang, Jun Yan, Yanfei Chen, Rujun Han, Zifeng Wang, Bhavana Dalvi Mishra, Rui Meng, Chun-Liang Li, Yizhu Jiao, Kaiwen Zha, Maohao Shen, Vishy Tirumalashetty, George Lee, Jiawei Han, Tomas Pfister, and Chen-Yu Lee. SkillOS: Learning skill curation for self-evolving agents, 2026.
- Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with SWE-Gym, 2024.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models, 2023.

Yifan Yang, Ziyang Gong, Weiquan Huang, Qihao Yang, Ziwei Zhou, Zisu Huang, Yan Li, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Yuqing Yang, Dongdong Chen, Xue Yang, and Chong Luo. SkillOpt: Executive strategy for self-evolving agent skills, 2026a.

Yutao Yang, Junsong Li, Qianjun Pan, Bihao Zhan, Yuxuan Cai, Lin Du, Jie Zhou, Kai Chen, Qin Chen, Xin Li, Bo Zhang, and Liang He. AutoSkill: Experience-driven lifelong learning via skill self-evolution, 2026b.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

A IMPLEMENTATION NOTES

Accepted skills are stored under versioned directories `skills/accepted/vXXXX/`: repo skills under `_repos/`, failure-mode skills under `_failure_modes/`, and general SWE seeds under `_general/swe/`. Training artifacts include `task_events.jsonl`, `repo_clusters.jsonl`, `failure_clusters.jsonl`, `evaluator_decisions.jsonl`, `evaluator_calibration.jsonl`, `promotion_decisions.jsonl`, `harness_policy.md`, `generator_policy.md`, and `evaluator_policy.md`.

B COMMAND SKELETONS

```
# SWEGym evolution loop, one source cell.
python scripts/run_swegym_skill_evo_loop.py \
  --run-name swegym_<model>_<harness>_hse_main \
  --provider openai \
  --provider-model <model-id> \
  --provider-api openai-completions \
  --concurrency <N>

# SWE-bench Verified target-cell evaluation with accepted skills.
python scripts/run_skill_evo_verified.py \
  --run-name verified_<source>_to_<target>_hse_final \
  --dataset swe-bench/swe-bench-verified@2 \
  --provider openai \
  --provider-model <target-model-id> \
  --all-verified \
  --concurrency <N>
```